

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1110

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (*BL5*)

Assessment Tool Weightage: 30%

QUESTIONS: 6

Total Marks:30

Annexure A

Reverse Engineering Task

Code of Conduct:

I N.Murali Sai(192372309) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Murali sai

Q1. Legacy E-Learning Management System

Scenario: An existing E-Learning Management System has large modules responsible for student registration, course management, content delivery, assignment evaluation, and result generation. User interface code and business rules are tightly coupled, making modifications difficult.

1. Analysis of the Existing System

- The system is built as a small number of large, “god” modules (Registration, Course Management, Content Delivery, Assignment Evaluation, Result Generation) — each handling many unrelated responsibilities, giving very low cohesion.
- UI code and business rules are tightly coupled inside the same module, so a change in a validation rule or a screen layout forces edits to the same file — violating the Single Responsibility Principle (SRP).
- There is no separation between how data is displayed, how business logic is processed, and how data is persisted, so the modules cannot be unit-tested or reused independently.
- Adding a new feature (e.g., a new assessment type) requires touching multiple unrelated modules, indicating high coupling and low maintainability/scalability.

2. Identified Classes, Responsibilities, Attributes and Relationships

Class	Layer	Key Attributes	Key Responsibilities (Methods)
Student	Domain	studentId, name, email	getProfile() — holds student data only
Course	Domain	courseId, title, credits	getSyllabus() — holds course data only
Enrollment	Domain	enrollId, status, date	links Student ↔ Course (association class)
Assignment / Result	Domain	assignId/resultId, dueDate/grade	hold assignment & result data
EnrollmentService	Service	—	enrollStudent(), dropCourse(), validateEligibility()
ContentDeliveryService	Service	—	uploadContent(), streamContent()
AssignmentEvaluationService	Service	—	evaluate(), applyRubric()
ResultGenerationService	Service	—	computeGrade(), publishResult()
*Controller classes	Presentation	—	accept UI input, delegate to services only
!*DAO interfaces	Data Access	—	save(), findById() against the OODBMS

Relationships: Student and Course are associated through Enrollment (an association/aggregation class); Assignment and Result are aggregated under Course; each Service depends on the Domain classes it manipulates and on the corresponding DAO interface (dependency, not inheritance), keeping persistence swappable.

3. Redesigned Layered Architecture

The monolithic modules are decomposed into four layers — Presentation (controllers), Business/Service (use-case orchestration), Domain (entities), and Data Access (DAO/OODBMS) — following the classic layered architecture pattern, so each class has one reason to change (SRP) and layers communicate only through well-defined interfaces (Dependency Inversion for persistence).

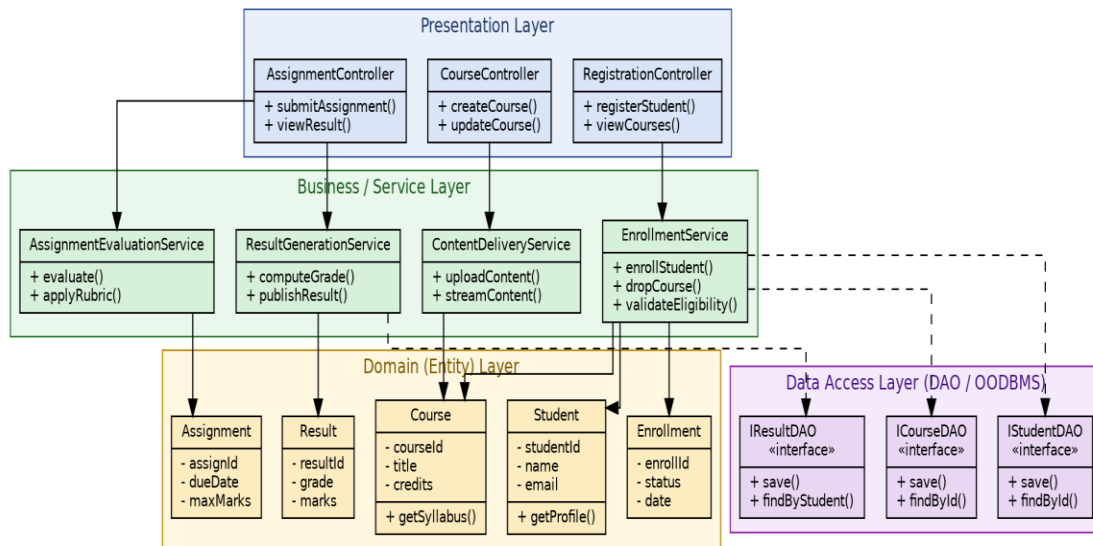


Fig 1.1 Layered Architecture - Redesigned E-Learning Management System

4. Improvement Explanation

Quality Attribute	How the Redesign Achieves It
Cohesion	Each class now has a single, focused responsibility (e.g., <code>ResultGenerationService</code> only computes/publishes results) instead of one module doing registration, content and evaluation together.
Coupling	Controllers depend only on Service interfaces, and Services depend only on DAO interfaces — UI, business rules and persistence are decoupled, so changing the database technology no longer touches business logic.
Maintainability	A change to an evaluation rule is isolated to <code>AssignmentEvaluationService</code> ; the Presentation and Data Access layers remain untouched, reducing regression risk.
Scalability	New features (e.g., a Certificate module) can be added as a new Service + DAO pair without modifying existing layers, supporting horizontal growth of the system.

Q2. Legacy Customer Support System

Scenario: A customer support application uses multiple conditional statements to handle support requests through email, chatbot, telephone, and social media channels. Adding a new support channel requires changes to several existing modules.

1. Object-Oriented Design Limitations

- Channel handling is implemented as a long if-else / switch chain inside a single class, so the class must be edited every time a channel is added — a direct violation of the Open/Closed Principle (OCP).
- Because channel-specific logic is procedural rather than polymorphic, the same conditional structure is duplicated wherever a ticket must be routed, increasing coupling and the risk of inconsistent behaviour.
- There is no common abstraction for a “support channel”, so the class cannot be extended through subtyping and unit tests must exercise the entire conditional block for every channel.

2. Reverse-Engineered Classes and Behavioural Pattern

Class / Interface	Type	Responsibility	Pattern Role
ISupportChannel	Interface	handleRequest(ticket)	Strategy interface
EmailChannelHandler	Concrete class	handles email tickets	Concrete Strategy
ChatbotChannelHandler	Concrete class	handles chatbot tickets	Concrete Strategy
TelephoneChannelHandler	Concrete class	handles phone tickets	Concrete Strategy
SocialMediaChannelHandler	Concrete class	handles social-media tickets	Concrete Strategy
SupportTicketContext	Class	holds current ISupportChannel, delegates handleRequest()	Strategy Context
SupportTicket	Domain class	ticketId, customer, issue, status	data passed to the strategy

3. Redesign — Strategy Pattern

The refactor replaces the conditional block with the Strategy behavioural pattern: each channel becomes a concrete implementation of ISupportChannel, and SupportTicketContext is configured with the appropriate strategy at runtime (e.g., chosen by the ticket's source field) instead of branching internally.

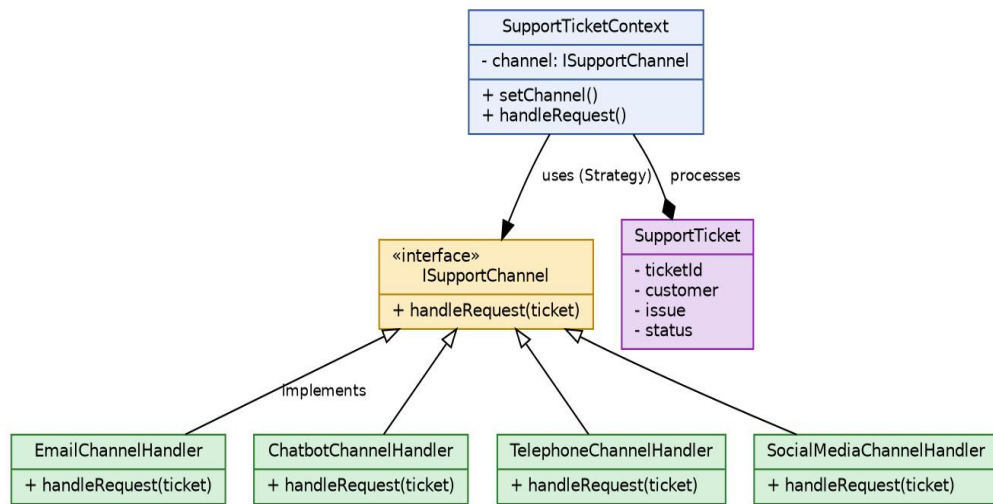


Fig 2.1 Strategy Pattern - Redesigned Customer Support Channel Handling

4. Improvement Explanation

Quality Attribute	How the Redesign Achieves It
Flexibility	The active channel handler can be swapped at runtime simply by assigning a different <code>ISupportChannel</code> implementation to the context.
Extensibility (OCP)	A new channel (e.g., WhatsApp) is added by writing one new class that implements <code>ISupportChannel</code> — no existing class is modified.
Loose Coupling	<code>SupportTicketContext</code> depends only on the <code>ISupportChannel</code> abstraction, not on any concrete channel class.
Maintainability	Each handler is independently testable and its logic is isolated from every other channel, eliminating the tangled conditional chain.

Q3. Legacy Banking Transaction System

Scenario: A banking application contains account management, transaction validation, fund transfer, loan processing, and transaction reporting within a few tightly coupled classes. Database operations are also embedded directly within business methods.

1. Analysis of the Existing System

- A handful of classes carry out account management, validation, transfer, loans and reporting together, giving each class multiple unrelated reasons to change (low cohesion).
- SQL/OODBMS calls are written directly inside business methods, so business logic cannot be tested without a live database connection and cannot be reused with a different persistence technology.
- Because responsibilities are not separated, a change to a validation rule risks breaking fund-transfer or reporting code that lives in the same class.

2. Identified Domain, Service and Data-Access Components

Component	Layer	Examples	Responsibility
Account, Transaction, Loan	Domain	accNo/balance, txnId/amount, loanId/principal	carry pure business data, no persistence logic
AccountService, TransactionService	Service	openAccount(), validateTxn(), postTxn()	enforce business rules
FundTransferService	Service (Facade)	transfer()	orchestrates AccountService + TransactionService
LoanProcessingService, ReportingService	Service	evaluateLoan(), generateStatement()	loan approval logic, statement generation
IAccountRepository, ITransactionRepository, ILoanRepository	Data Access	save(), findById()	abstract OODBMS operations behind an interface

3. Redesigned Layered Architecture with Repository Pattern

Business rules are extracted into Service classes that depend only on Repository interfaces (the Repository/DAO pattern), while FundTransferService acts as a light Facade that coordinates AccountService and TransactionService for a single logical operation, keeping database code entirely out of the business layer.

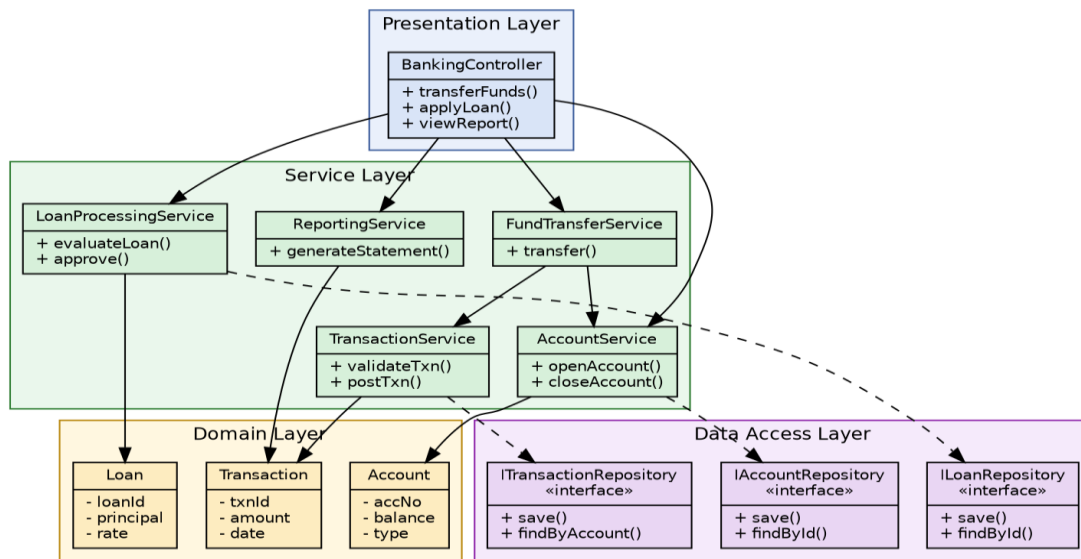


Fig 3.1 Layered Architecture - Redesigned Banking Transaction System

4. Improvement Explanation

Quality Attribute	How the Redesign Achieves It
Separation of Concerns	Domain data, business rules and persistence now live in three distinct layers instead of being mixed in one class.
Testability	Services can be unit-tested by injecting a mock IAccountRepository / ITransactionRepository, with no real database required.
Maintainability	A change to how a loan is evaluated is confined to LoanProcessingService and never touches persistence or reporting code.
Scalability	The OODBMS implementation behind each repository interface can be replaced or sharded without any change to the Service or Domain layers.

Q4. Legacy Hotel Reservation System

Scenario: A hotel reservation application directly performs OODBMS operations from reservation screens. The same database queries for rooms, guests, bookings, and payments are repeated across multiple modules.

1. Problems in the Existing Architecture

- Reservation screens (presentation-layer code) call the OODBMS directly, so UI classes are tightly coupled to persistence details and cannot be reused or tested without a live database (coupling).
- Identical room/guest/booking/payment queries are copy-pasted in several screens — a classic duplication problem that makes a schema change error-prone, since every copy must be updated.
- There is no single place responsible for persistence management, so transactions spanning room + booking + payment updates are hard to keep consistent.

2. Reverse-Engineered Entities and Access Components

Component	Layer	Examples	Responsibility
Room, Guest, Booking, Payment	Domain	roomNo/type, guestId, bookingId, paymentId	pure entity data
BookingService, PaymentService	Service	createBooking(), processPayment()	business rules for reservation & payment
IRoomRepository, IGuestRepository, IBookingRepository, IPaymentRepository	Repository (interface)	findAvailable(), save(), findByGuest()	single abstraction point for every query
*RepositoryOODBMS	Repository (impl.)	concrete OODBMS access	the only classes that talk to the database

3. Redesign — DAO / Repository Pattern

Every OODBMS query for Room, Guest, Booking and Payment is centralised behind one Repository interface per entity; reservation screens now call BookingService/PaymentService, which in turn call the repositories — eliminating duplicated queries and removing persistence code from the presentation layer entirely.

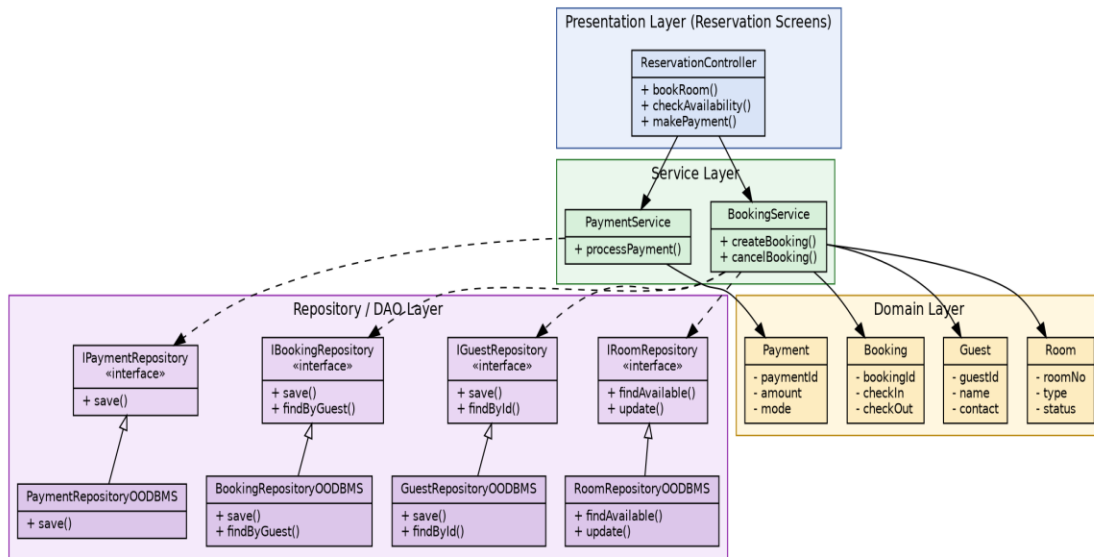


Fig 4.1 Repository / DAO Pattern - Redesigned Hotel Reservation System

4. Improvement Explanation

Quality Attribute	How the Redesign Achieves It
Modularity	Each entity's persistence logic lives in exactly one repository class instead of being scattered across screens.
Reusability	BookingService and its repositories can be reused by a future mobile app or admin console without duplicating queries again.
OODBMS Integration	Repository interfaces isolate the rest of the system from OODBMS-specific APIs, so the underlying database can evolve independently.
Consistency	Multi-entity operations (e.g., booking + payment) can be coordinated inside a single Service method, simplifying transaction management.

Q5. Legacy Transportation Management System

Scenario: A transportation application contains separate modules for buses, trains, taxis, and rental vehicles. Object creation logic for each transportation type is duplicated across several parts of the application, making it difficult to introduce new transportation services.

1. Analysis of the Existing System

- Bus, Train, Taxi and RentalVehicle are handled as separate, largely independent modules with no shared abstraction, so common behaviour (fare calculation, capacity) is re-implemented rather than reused.
- The code that decides which concrete class to instantiate is copy-pasted wherever a transport object is needed, so adding a new transport type means editing every one of those call sites — violating OCP.

2. Identified Abstractions and Object Relationships

Class / Interface	Type	Responsibility	Pattern Role
Transport	Interface	calculateFare(), getCapacity()	common product abstraction
Bus, Train, Taxi, RentalVehicle	Concrete classes	implement Transport for each mode	Concrete Product
TransportFactory	Abstract class	createTransport()	Factory Method — declares the creation hook
BusFactory, TrainFactory, TaxiFactory, RentalVehicleFactory	Concrete factories	override createTransport()	Concrete Creator
TransportBookingService	Client	bookTransport(type)	works only against Transport and TransportFactory

3. Redesign — Factory Method Pattern

All four transport types are unified under a common Transport interface, and object creation is moved out of client code into one factory per type (Factory Method). The client (TransportBookingService) asks a TransportFactory for a Transport instance and never references a concrete class directly — the duplicated creation logic collapses into a single, extensible hierarchy.

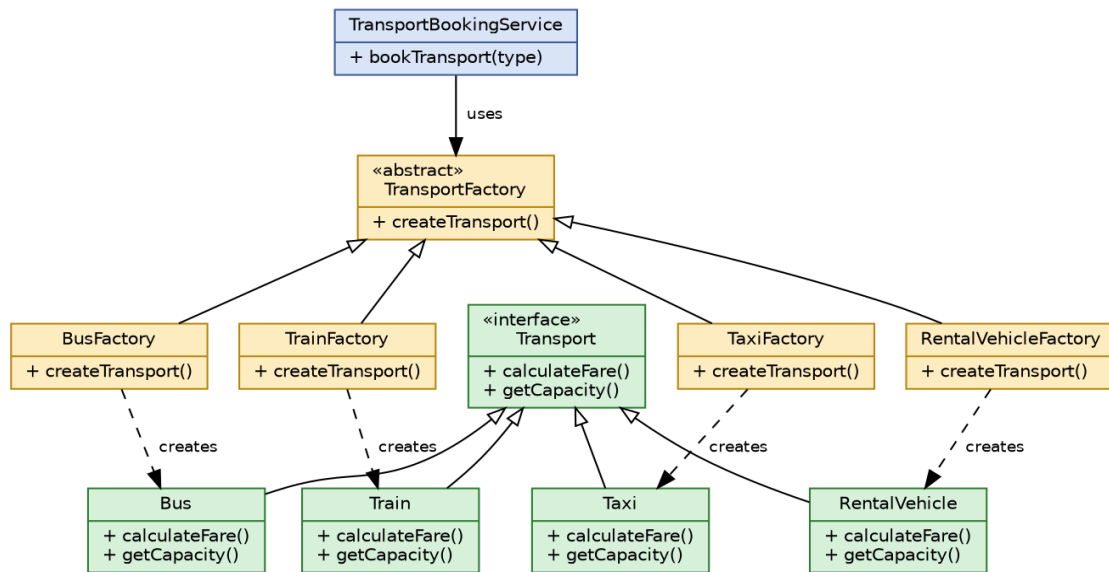


Fig 5.1 Factory Method Pattern - Redesigned Transportation Management System

4. Improvement Explanation

Quality Attribute	How the Redesign Achieves It
Extensibility	A new mode (e.g., Metro) is added by creating a Metro class + MetroFactory — existing factories and client code stay untouched (Open/Closed).
Scalability	New transport types plug into the same interface, so the booking workflow scales to any number of modes without structural change.
Code Reuse	Common behaviour required of every transport is declared once in the Transport interface instead of being duplicated per module.
Loose Coupling	TransportBookingService depends only on the abstract Transport and TransportFactory types, never on concrete vehicle classes.

Q6. Legacy Healthcare Monitoring System

Scenario: A healthcare monitoring application collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform multiple unrelated tasks and directly depend on concrete implementations of monitoring devices and storage components.

1. SOLID Violations Identified

- **SRP violation:** a single PatientMonitorService both reads sensors, generates alerts and stores records — three unrelated reasons to change.
- **OCP violation:** adding a new device (e.g., a glucose monitor) or a new storage medium requires editing this same class rather than extending it.
- **DIP violation:** the high-level monitoring logic depends directly on concrete device and storage classes instead of on abstractions, so it cannot be tested or reconfigured without touching those concrete classes.

2. Redesigned Classes and Abstractions

Class / Interface	Type	Responsibility	SOLID Principle Applied
IMonitoringDevice	Interface	readSensor()	DIP / OCP — new devices implement this
ECGDevice, SpO2Device	Concrete classes	device-specific reading logic	OCP
IRecordStorage	Interface	save(record), fetch(id)	DIP — storage medium is abstracted
RelationalRecordStorage, CloudRecordStorage	Concrete classes	persist PatientRecord	OCP
IAAlertNotifier	Interface	notify(alert)	DIP — notification channel is abstracted
PatientMonitorService	Service (high-level)	collectReading(), evaluateReading()	SRP — depends only on the 3 interfaces above
ReportGenerator	Service	generateReport()	SRP — separated out of the monitor service

3. Redesign — Abstractions Enforcing SOLID

PatientMonitorService is refactored to depend only on IMonitoringDevice, IRecordStorage and IAAlertNotifier (constructor-injected), satisfying DIP; report generation is extracted into its own ReportGenerator class, satisfying SRP; and new devices, storage back-ends or notification channels are added purely by implementing the relevant interface, satisfying OCP.

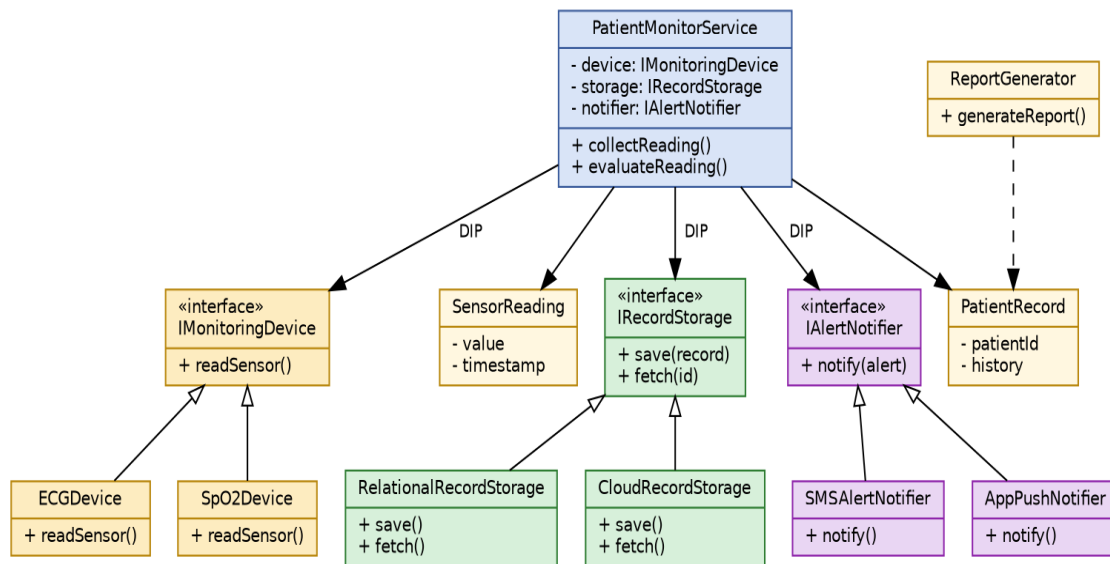


Fig 6.1 SOLID-Compliant Redesign - Healthcare Monitoring System (DIP via Abstractions)

4. Improvement Explanation

Quality Attribute	How the Redesign Achieves It
Cohesion	Each class now has one job — reading, storing, notifying or reporting — instead of PatientMonitorService doing all four.
Loose Coupling	PatientMonitorService is wired to interfaces, not concrete devices/storage, so those can change independently.
Flexibility / OCP	Supporting a new sensor or database is an extension (new class implementing an interface), never a modification of existing code.
Testability	Unit tests can inject fake IMonitoringDevice / IRecordStorage / IAlertNotifier implementations, removing the need for real hardware or a database.
Maintainability	Bug fixes or upgrades to alerting logic are isolated to the relevant notifier class and cannot ripple into sensor or storage code.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation